



Institut Supérieur d'Informatique  
et de Multimédia de Gabès

# Programmation Orientée Objet 2

## JAVA

**Sofiane HACHICHA**

**[sofiane.hachicha@isimg.tn](mailto:sofiane.hachicha@isimg.tn)**

**CPI2**

**2025/2026**



# Objectifs du cours

**POO : Java avancé**



- **Comprendre et utiliser les exceptions**
- **Représenter et manipuler les collections d'objets.**
- **Gérer les flux d'entrées et sorties avec Java.**
- **Proposer des interfaces graphiques en utilisant l'API Swing.**
- **Manipuler les bases de données.**





Institut Supérieur d'Informatique  
et de Multimédia de Gabes

# Contenu

**P00 : Java avancé**



## Gestion des exceptions

Sofiane HACHICHA  
ISIMG 2025 - 2026





Institut Supérieur d'Informatique  
et de Multimédia de Gabes

**P00 : Java avancé**



# Chapitre

**1**

# Gestion des Exceptions



# 1- Exceptions

## 1.1 Introduction (1/3)

- Souvent, un programme doit traiter des **situations inattendues (exceptionnelles)** en dehors de sa tâche principale.
- Une situation exceptionnelle peut être assimilée à une **erreur** qui génère une **interruption** gérée habituellement par le système d'exploitation.
- Le mécanisme de gestion d'erreur le plus répandu est celui des **exceptions**.
- **Exemples d'erreurs/exceptions courantes :**
  - Accès non autorisé à une zone mémoire, division par zéro, débordement d'indices dans un tableau, etc.





# 1- Exceptions

## 1.1 Introduction (2/3)

### ● Exemple :

```
public class TestException {  
    public static void main(String[] args) {  
        int i = 3;  
        int j = 0;  
        System.out.println("résultat = " + (i / j));  
        System.out.println("Cette instruction ne sera jamais exécutée !");  
    }  
}
```

- Lors de l'exécution, la JVM va générer l'erreur (exception) **java.lang.ArithmeticException: / by zero** et le programme **TestException** sera interrompu.





# 1- Exceptions

## 1.1 Introduction (3/3)

- Une exception est un **événement**, se produisant lors de l'exécution d'un programme, qui interrompt l'enchaînement normal des instructions.
- L'**objectif** est de **gérer ces exceptions** par **le programme lui-même** en les interceptant (en les **capturant**).
- Le **principe** consiste à **repérer les morceaux de code** (par exemple, une division par zéro) qui pourraient générer une exception, de **capturer l'exception** correspondante et enfin de **la traiter**, c'est-à-dire d'afficher un message personnalisé et de continuer l'exécution.
- En Java, **Exception** est une classe étendant la classe **Throwable**



# 1- Exceptions

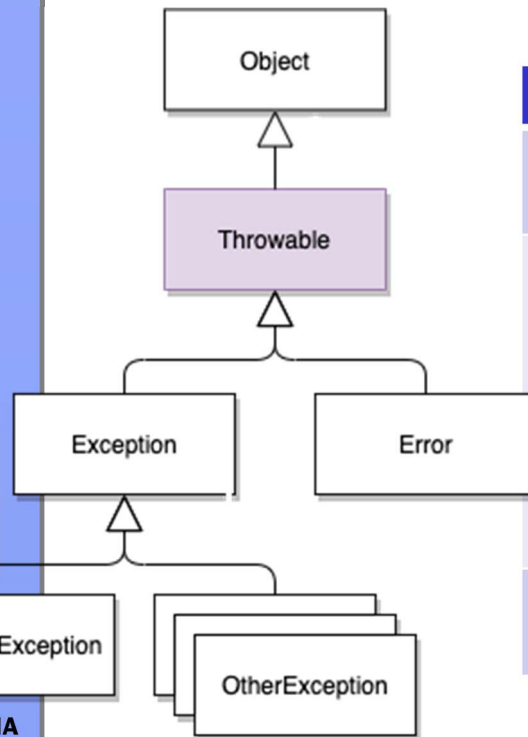
## 1.2 Arbre des exceptions (1/6)

P.O.O : Java avancé



Sofiane HACHICHA  
ISIMG 2025 - 2026

● Classe **Throwable** : classe de base de toutes les exceptions et étendant directement la classe « **Object** ».



| Méthodes               |                                                                                                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| String getMessage()    | Retourne le message de l'exception                                                                                                                                                |
| void printStackTrace() | Affiche l'exception avec l'état de la pile (« stack ») : affiche l'exception avec d'autres détails comme le numéro de ligne et le nom de la classe où l'exception s'est produite. |
| Throwable getCause()   | Retourne l'origine (la cause racine) de l'exception                                                                                                                               |



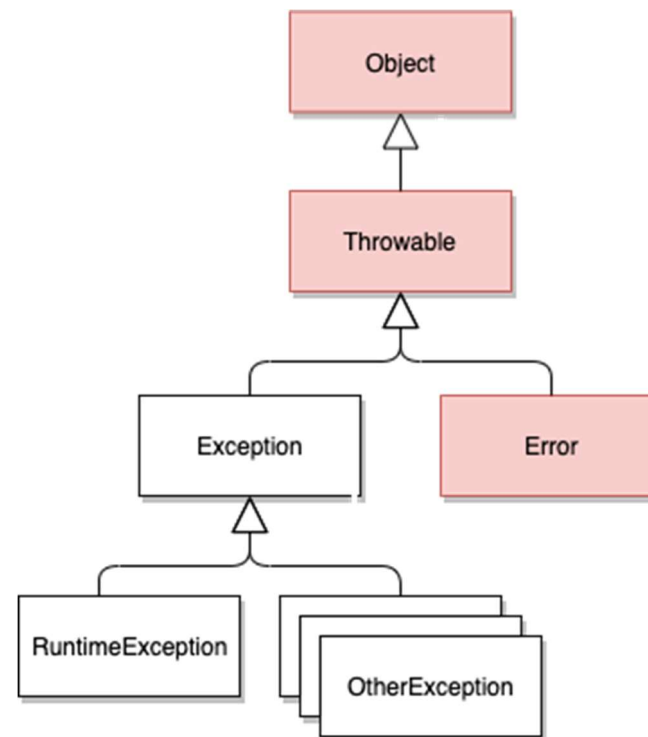


# 1- Exceptions

## 1.2 Arbre des exceptions (2/6)



- Classe **Error** représentant une **erreur grave** en provenance de la JVM ou d'un de ses composants
  - Cette erreur entraine un **arrêt du programme**



**Vous ne devez pas étendre la classe Error**

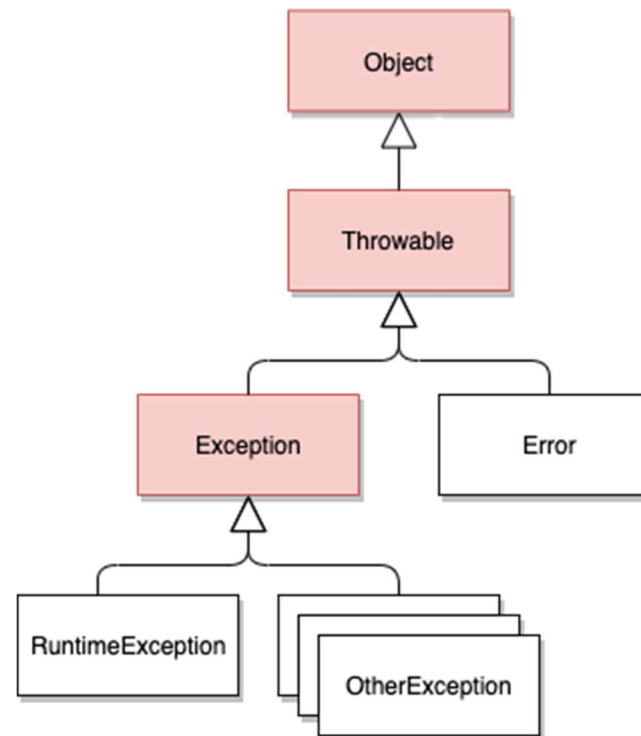


# 1- Exceptions

## 1.2 Arbre des exceptions (3/6)



- Classe **Exception** : représentant les erreurs classiques remontées le plus souvent par les méthodes.
- Ces exceptions doivent être interceptées par le bloc « **try catch** »



# 1- Exceptions

## 1.2 Arbre des exceptions (4/6)



● Classe **RuntimeException** dont les exceptions pouvant être levées sans déclaration via **throws** dans la signature de la méthode. Ce sont les **unchecked exceptions** (ou exceptions implicites)

● Tentative de création d'un tab de taille négative

● **NegativeArraySizeException**

● Impossibilité de convertir une chaîne en nombre

● **NumberFormatException**

● Dépassement de limite d'un tableau

● **ArrayIndexOutOfBoundsException**

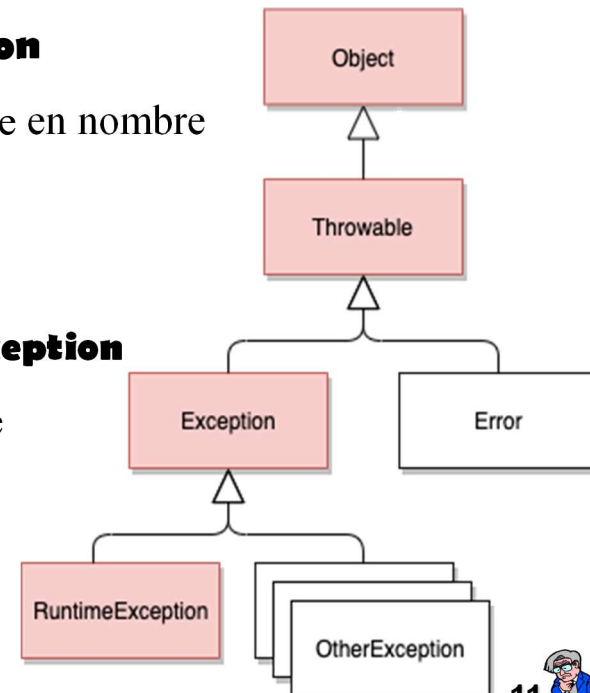
● Tentative de forçage de type illégale

● **ClassCastException**

● Division par zéro pour les entiers

● **ArithmeticException**

● ...



# 1- Exceptions

## 1.2 Arbre des exceptions (5/6)



### Exception **personnalisée**

- Pour créer de nouvelles exceptions, il suffit d'étendre la classe « **Exception** »

```
public class UserNotFoundException extends Exception{  
    public UserNotFoundException() {  
        super("Utilisateur non trouvé");  
    }  
}
```

Par convention, les exceptions sont  
suffixées par le mot « Exception »



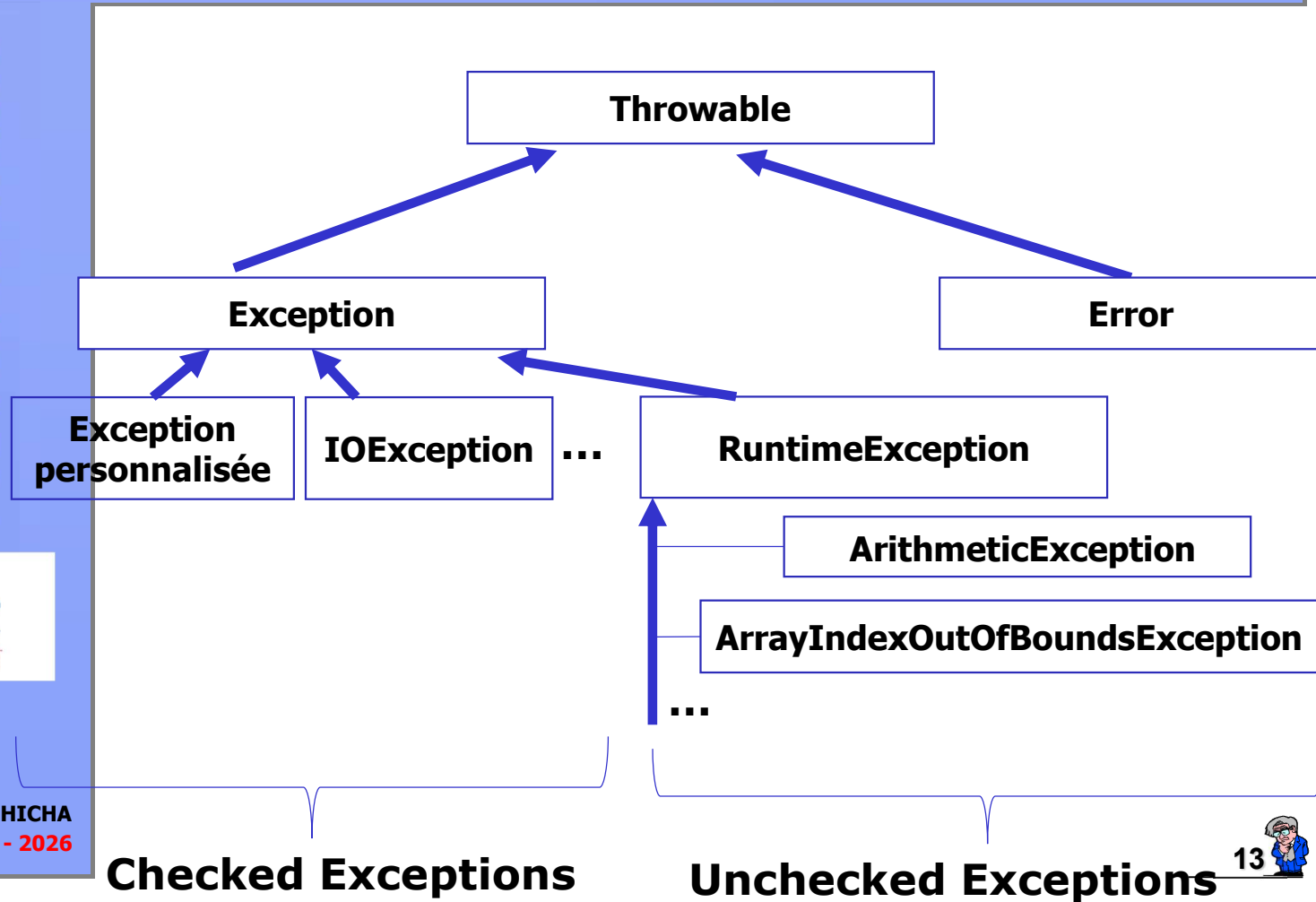
# 1- Exceptions

## 1.2 Arbre des exceptions (6/6)

POO : Java avancé



Sofiane HACHICHA  
ISIMG 2025 - 2026



# 1- Exceptions

## 1.3 Gestion des exceptions en Java (1/5)



### ● Vocabulaire

- Lever ou lancer (**throw**) une exception consiste à signaler cette erreur.
- Capturer ou attraper (**catch**) une exception permet d'exécuter les actions nécessaires pour la traiter.
- Java exige qu'une méthode susceptible de **lever** une exception (hormis les **Error** et **RuntimeException**) indique qu'elle doit être l'action à réaliser.
  - Sinon, il y a erreur de compilation.
- Le programmeur a le choix entre :
  - écrire un bloc **try / catch** pour traiter l'exception,
  - laisser remonter l'exception au bloc appelant grâce à un **throws**.
    - Toute méthode susceptible de déclencher une **checked exception** qu'elle ne traite pas localement doit mentionner son type dans une clause **throws** figurant dans son en-tête.

# 1- Exceptions

## 1.3 Gestion des exceptions en Java (2/5)



- On délimite un ensemble d'instructions susceptibles de déclencher une exception par un bloc **try**.
- Le bloc **try** est exécuté jusqu'à ce qu'il se termine avec succès ou bien qu'une exception soit levée.
- Dans ce dernier cas, les clauses **catch** sont examinées l'une après l'autre dans le but d'en trouver une qui traite **cette classe d'exceptions** (ou une **superclasse**).
- Les clauses **catch** doivent donc **traiter les exceptions de la plus spécifique à la plus générale**.
  - La présence d'une clause **catch** qui intercepte une classe d'exceptions avant une clause qui intercepte une sous-classe d'exceptions déclenche une erreur de compilation.
- Si une clause **catch** convenant à cette exception a été trouvée et le bloc exécuté, l'exécution du programme **reprend** son cours.



# 1- Exceptions

## 1.3 Gestion des exceptions en Java (3/5)



- Il est possible d'utiliser plusieurs **mêmes lignes de code** dans **plusieurs clauses catch**

```
try {
    // traitements pouvant lever les exceptions
}
catch(ExceptionType1 e1) { // Traitement de l'exception }
catch(ExceptionType2 e2) { // Traitement de l'exception }
catch(ExceptionType3 e3) { // Traitement de l'exception }
```

- Depuis la version **Java 7**, cette portion de code est simplifiée
  - il suffit de déclarer les exceptions dans une même clause catch en les séparant par le caractère "|".

```
try {
    // traitements pouvant lever les exceptions
}
catch(ExceptionType1 | ExceptionType2 | ExceptionType3 ex)
{ // Traitement de l'exception }
```





# 1- Exceptions

## 1.3 Gestion des exceptions en Java (4/5)



- Si une exception n'est pas immédiatement capturée par un bloc **catch**
  - elle se propage en remontant la pile d'appels des méthodes, jusqu'à être traitée.
- Si une exception n'est jamais capturée
  - elle se propage jusqu'à la méthode `main()`, ce qui pousse l'interpréteur Java à afficher un message d'erreur et à s'arrêter.
  - l'interpréteur Java affiche un message identifiant :
    - l'exception,
    - la méthode qui l'a causée,
    - la ligne correspondante dans le fichier.



# 1- Exceptions

## 1.3 Gestion des exceptions en Java (5/5)



- Pour indiquer qu'une méthode peut **lever** une exception, il faut utiliser le mot clé **throws**, suivi de la classe d'exception, dans la déclaration de la méthode dans laquelle l'exception est susceptible de se manifester.
  - La classe de l'exception indiquée peut être une **super-classe** de l'exception effectivement générée.
  - Une même méthode peut "laisser remonter" plusieurs types d'exceptions (séparés par des ,).
- Pour lever l'exception, le mot clé **throw** doit être utilisé suivi d'une instance de la classe d'exception.
- La levée de l'exception entraîne une interruption de la méthode.





# 1- Exceptions

## 1.4 Exemple 1 (1/2)



```
class Point {  
    private int x, y;  
    public Point(int x, int y) throws ErrConst {  
        if ((x < 0) || (y < 0)) throw new ErrConst();  
        this.x = x; this.y = y;  
    }  
    public void deplace (int dx, int dy) throws ErrDepl {  
        if (((x + dx) < 0) || ((y + dy) < 0)) throw new ErrDepl();  
        x = x + dx; y = y + dy;  
    }  
    public void affiche() {  
        System.out.println(" Coord : " + x + " " + y);  
    }  
}  
  
class ErrConst extends Exception { }  
class ErrDepl extends Exception { }
```





# 1- Exceptions

## 1.4 Exemple 1 (2/2)



```
public class Except1 {  
    public static void main (String args[])  
    {  
        try{  
            Point a = new Point(1,4);  
            a.affiche();  
            a.deplace(-3,5);  
            a = new Point (-3,5);  
            a.affiche();  
        }  
        catch (ErrConst e1){  
            System.out.println(" Erreur construction " );  
            System.exit(-1);  
        }  
        catch (ErrDepl e2) {  
            System.out.println(" Erreur déplacement " );  
            System.exit(-1);  
        }  
    }  
}
```

Résultat d'affichage



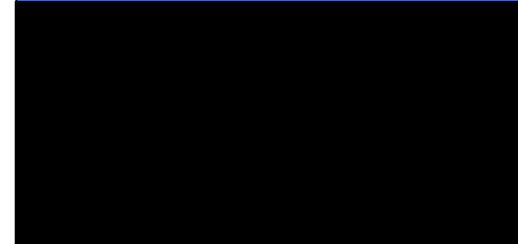
# 1- Exceptions

## 1.4 Exemple 2



```
public class Except2 {
    public static void main (String args[]) {
        System.out.println(" Avant bloc try ");
        try { Point a = new Point(1,4);
            a.affiche();
            a = new Point (-3,5);
            a.deplace(-3,5);
            a.affiche(); }
        catch (ErrConst e1)
            { System.out.println(" Erreur construction " ); }
        catch (ErrDepl e2)
            { System.out.println(" Erreur déplacement " ); }
        System.out.println(" Apres bloc try ");
    }
}
```

### Résultat d'affichage





# 1- Exceptions

## 1.4 Exemple 3



```
public class Except3 {  
    public static void main (String args[])  
    {  
        try{ Point a = new Point(1,4);  
            a.deplace(2,5);  
        }  
        catch (ErrConst e)  
        {  
            System.out.println(" Erreur construction " );  
            System.exit(-1);  
        }  
        System.out.println(" Au revoir " );  
    }  
}
```

**La construction suivante serait rejetée par le compilateur**





# 1- Exceptions

## 1.4 Exemple 4 (1/2)



```
class Point
{
    private int x, y;
    public Point(int x, int y) throws ErrConst
    { if ((x < 0) || (y < 0)) throw new ErrConst(x,y);
      this.x = x; this.y = y;
    }
    public void affiche()
    { System.out.println(" Coord : " + x + " " + y);}
}

class ErrConst extends Exception {
    public int abs, ord;
    ErrConst (int abs, int ord)
    {this.abs = abs; this.ord = ord;}
}
```



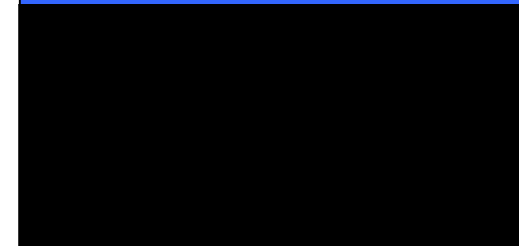
# 1- Exceptions

## 1.4 Exemple 4 (2/2)



```
public class Except4 {  
    public static void main (String args[])  
    { try { Point a = new Point(1,4);  
        a.affiche();  
        a = new Point (-3,5);  
        a.affiche();  
    }  
    catch (ErrConst e)  
    { System.out.println("Erreur construction Point" );  
      System.out.println("Coord souhaitees" + e.abs + " " + e.ord);  
      System.exit(-1); }  
    }  
}
```

Résultat d'affichage







# 1- Exceptions

## 1.4 Exemple 5 (1/2)



```
class Point
{
    private int x, y;
    public Point(int x, int y) throws ErrConst
    {
        if ((x < 0) || (y < 0)) throw new ErrConst("Erreur construction avec
                                                    coordonnees"+ x + " " + y);

        this.x = x; this.y = y;
    }
    public void affiche()
    { System.out.println(" Coord : " + x + " " + y); }
}

class ErrConst extends Exception {
    ErrConst (String mes)
    { super(mes); }
}
```



# 1- Exceptions

## 1.4 Exemple 5 (2/2)



```
public class Except5 {  
    public static void main (String args[])  
    { try{ Point a = new Point(1,4);  
        a.affiche();  
        a = new Point (-3,5);  
        a.affiche();  
    }  
    catch (ErrConst e)  
    { System.out.println( e.getMessage());  
      System.exit(-1);  }  
    }  
}
```

Résultat d'affichage



# 1- Exceptions

## 1.4 Exemple 6



```
int f()
{
    int n ;
    try {
        float x;
        ...
    }
    catch ( E1 e )
    {
        ...

        return 0;
    }
    ...
}
```

// ici, on n'a pas accès à x (bloc disjoint de celui-ci)

// mais on a accès à n (bloc englobant)

// en cas d'exception, on sort de f

// sans exécuter ces instructions



# 1- Exceptions

## 1.5 Choix du gestionnaire d'exception (1/4)



```
class ErrPoint extends Exception { }
class ErrConst extends ErrPoint { }
class ErrDepl extends ErrPoint { }
```

```
void f() throws ErrConst, ErrDepl
{ ..... throw new ErrConst();
  .....throw new ErrDepl();
}
```

Dans un programme utilisant la **méthode f**, on peut gérer les exceptions qu'elle est susceptible de déclencher des façons suivantes :

```
try {
    ... // on suppose qu'on utilise f
}
catch (ErrPoint e)
{ // on traite ici à la fois les exceptions de type ErrConst et celles de type ErrDepl
}
```



# 1- Exceptions

## 1.4 Choix du gestionnaire d'exception (2/4)



```
try {  
    ... // on suppose qu'on utilise f  
}  
catch (ErrConst e)  
{  
    // on traite ici uniquement les exceptions de type ErrConst  
}  
catch (ErrDepl e)  
{  
    // et ici, uniquement celles de type ErrDepl  
}
```



# 1- Exceptions

## 1.5 Choix du gestionnaire d'exception (3/4)



```
try {  
    ...           // on suppose qu'on utilise f  
}  
catch (ErrConst e)  
{  
    // on traite ici uniquement les exceptions de type ErrConst  
}  
catch (ErrPoint e)  
{  
    // et ici, toutes celles de type ErrPoint ou dérivé (autre que ErrConst)  
}
```



# 1- Exceptions

## 1.5 Choix du gestionnaire d'exception (4/4)



Soit la déclaration suivante :

```
try
{ ..... }
catch (ErrPoint e)
{ ..... }
catch (ErrConst e)
{ ..... }
```

On obtiendrait une **erreur de compilation** due à ce que le gestionnaire ErrConst n'a plus désormais aucune chance d'être atteint.





# 1- Exceptions

## 1.6 Redéclenchement d'une exception (1/2)



```
class Point
{
    private int x, y;
    public Point(int x, int y) throws ErrConst
    {
        if ((x < 0) || (y < 0)) throw new ErrConst();
        this.x = x; this.y = y;
    }
    public void f() throws ErrConst
    { try { Point p = new Point(-3,2); }
      catch (ErrConst e)
      { System.out.println(" dans catch ( ErrConst ) de f " );
        throw e;      // on repasse l'exception à un niveau supérieur
      }
    }
}
```





# 1- Exceptions

## 1.6 Redéclenchement d'une exception (2/2)



```
class ErrConst extends Exception {}
public class Redec1
{
    public static void main (String args[])
    { try
        { Point a = new Point(1,4);
          a.f();
        }
        catch (ErrConst e)
        { System.out.println(" dans catch ( ErrConst ) de main" ); }
        System.out.println(" Apres bloc try main" );
    }
}
```

Résultat d'affichage

Cette possibilité de **redéclenchement d'une exception** s'avère très précieuse lorsque l'on ne peut résoudre localement qu'une partie du problème posé.



# 1- Exceptions

## 1.7 Exemple 7 (1/2)



```
class Point
{
    private int x, y;
    public Point(int x, int y) throws ErrConst
    {
        if ((x < 0) || (y < 0)) throw new ErrConst();
        this.x = x; this.y = y;
    }
    public void f() throws ErrBidon
    { try { Point p = new Point(-3,2); }
      catch (ErrConst e)
      { System.out.println(" dans catch (ErrConst) de f" );
        throw new ErrBidon();           // on lance une nouvelle exception
      }
    }
}
```





# 1- Exceptions

## 1.7 Exemple 7 (2/2)



```
class ErrConst extends Exception {}
class ErrBidon extends Exception {}
public class Redec2
{
    public static void main (String args[])
    { try
        { Point a = new Point(1,4);
          a.f(); }
      catch (ErrConst e)
        { System.out.println(" dans catch (ErrConst) de main" );}
      catch (ErrBidon e)
        { System.out.println(" dans catch (ErrBidon) de main" );}
      System.out.println(" Apres bloc try main" );
    }
}
```

Résultat d'affichage



# 1- Exceptions

## 1.8 Bloc finally



- Le bloc **finally**
  - il permet au programmeur de définir un ensemble d'instructions qui sont toujours exécutées, que l'exception soit :
    - levée ou non,
    - capturée ou non
- Le bloc finally s'exécute même si
  - le bloc en cours d'exécution (try ou catch selon les cas) contient un **return** ou un **break**.
    - Dans ce cas, le bloc finally est exécuté juste avant le branchement effectué par l'une de ces instructions.
  - La seule instruction qui peut faire qu'un bloc finally ne soit pas exécuté est `System.exit()`.
- Le bloc finally est optionnel et il doit obligatoirement être placé après le dernier gestionnaire.



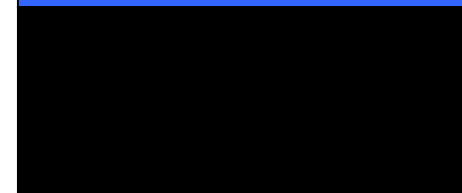
# 1- Exceptions

## 1.9 Exemple 8



```
class Except extends Exception { }
public class TestFinally
{
    public static void f(int n) // pas de throws
    {
        try
        {
            if (n!=1) throw new Except();
        }
        catch (Except e)
        {
            System.out.println(" catch dans f - n = " + n );
        }
        finally
        {
            System.out.println(" dans finally - n = " + n );
        }
    }
    public static void main (String args[])
    {
        f(1);
        f(2);
    }
}
```

### Résultat d'affichage



# 1- Exceptions

## 1.9 Exemple 9



```
public class ExcStd
{
    public static void test(int n , int i){
        if (n < 0) throw new NegativeArraySizeException();
        int t[]= new int[n];
        t[i] = 12;
    }
    public static void main (String args[]){
        try{
            int n =2,i=10;
            test(n,i);
            System.out.println(" fin normale " );
        } catch (ArrayIndexOutOfBoundsException e)
        { System.err.println("Exception indice tab : " + e.getMessage()); }
        finally{ System.out.println("Bloc Finally"); }
    }
}
```

Résultat d'affichage





# 1- Exceptions

## 1.9 Exemple 10



```
import java.util.*;

class LargeListException extends RuntimeException {
    public LargeListException(String message) {
        super(message);
    }
}

class TestException extends Exception {
    public TestException(String message) {
        super(message);
    }
}

public class Stock {
    public void check(ArrayList<String> products) {
        if (products.size() > 10) {
            throw new LargeListException("La liste ne peut pas dépasser 10 produits!");
        }
    }
}
```





# 1- Exceptions

## 1.9 Exemple 10



```
public void empty(ArrayList<String> products) throws TestException {  
    if (products.isEmpty()) {  
        throw new TestException("La liste ne peut pas être vide!");  
    }  
}  
  
public static void main(String[] args) throws TestException {  
    try {  
        Stock stock = new Stock();  
        ArrayList<String> products = new ArrayList<>();  
        for (int i = 0; i < 20; i++)  
            products.add("Produit" + i);  
        stock.empty(products);  
        stock.check(products);  
    } catch (LargeListException e) { e.printStackTrace(); }  
    System.out.println("Fin du programme");  
} }
```





# 1- Exceptions

## 1.9 Exemple 10



Résultat d'affichage



# 1- Exceptions

## 1.10 Instruction « try-with-resources » (1/3)



- Des ressources comme des fichiers, des flux, des connexions, ... doivent être fermées explicitement par le développeur pour libérer les ressources sous-jacentes qu'elles utilisent.
  - Utiliser un bloc **try / finally** pour garantir leur fermeture.
  - **Risque d'oubli de fermeture → fuite de ressources.**
- Depuis **Java 7**, l'instruction « **try-with-resources** » permet de définir une ressource qui sera automatiquement fermée à la fin de l'exécution du bloc de code de l'instruction.
- Tous les objets qui implémentent l'interface **java.lang.AutoCloseable** peuvent être utilisés dans une instruction de type try-with-resources.
  - InputStream, OutputStream, Reader, Writer (java.io), Connection, Statement, ResultSet (java.sql) ...



# 1- Exceptions

## 1.10 Instruction « try-with-resources » (2/3)



- L'interface `AutoCloseable` ne définit qu'une seule méthode abstraite : la méthode **`close()`**
  - c'est ce qui garantira à l'instruction « try-with-resources » qu'elle pourra forcer la fermeture de la ressource.
  - Si cette instruction est utilisée sur un type n'implémentant pas cette interface, une erreur de compilation sera produite.
- Les ressources à libérer sont positionnées entre parenthèses et directement à la suite du mot clé `try`.

```
// Une seule ressource automatiquement gérée
try ( FileInputStream fis = new FileInputStream( "Fich.txt" ) ) {
    // Ici on manipule le fichier
} // Appel à la méthode close automatique
```



# 1- Exceptions

## 1.10 Instruction « try-with-resources » (3/3)



```
// Deux ressources automatiquement gérées
try ( FileInputStream fis = new FileInputStream( "Fich1.txt" );
      FileOutputStream fos = new FileOutputStream( "Fich2.txt" ) ) {
    // Ici on manipule les deux fichiers
} // Appel aux méthodes close automatique
```

Avec **Java 9**, il est possible de **déclarer une variable** correspondant à une ressource à libérer automatiquement en dehors du bloc try.

```
FileInputStream fis = new FileInputStream( "Fich.txt" );
// Une seule ressource automatiquement gérée
try ( fis ) {
    // Ici on manipule le fichier
} // Appel à la méthode close automatique
```



# 1- Exceptions

## 1.11 Assertion (1/2)



- Une assertion permet de vérifier une condition considérée comme vraie.
  - Si cette condition est **vraie**, alors l'assertion sera **muette**.
  - Si cette condition est **fausse**, alors une **erreur** sera produite et la **JVM** lève une unchecked Exception **AssertionError**.
- Une assertion s'introduit via le mot clé **assert** et elle est suivie de la condition à vérifier et éventuellement d'un message d'erreur.
  - Deux syntaxes :
    - `assert condition;`                      `assert i==j;`
    - `assert condition : msg;`              `assert i==j : "i n'est pas égal à j";`
- Par défaut, les assertions ne sont pas exécutées, il faut lancer :
  - soit l'option court : **java -ea** (enable assert)
  - soit l'option longue : **java -enableassertions**



# 1- Exceptions

## 1.11 Assertion (2/2)



```
class AssertTest {  
    public static void main(String args[]) {  
        assert args.length > 2 && args.length < 4 : "Parametres Invalides (" + args.length + ")";  
        System.out.println("->" + args[2]);  
    }  
}
```

✓ Quel est le résultat de chaque commande suivante ?

**% java AssertTest**

**% java -ea AssertTest**

**% java -ea AssertTest 1 2 xy**